Parte 1 — Reconocer la estructura adecuada

1. **Un editor de texto guarda las acciones recientes para permitir deshacer primero la última acción realizada:**
    - **Estructura:** Pila (LIFO)1.
    - **Justificación:** Sigue el principio *Last-In, First-Out* (último en entrar, primero en salir), donde la acción más reciente se coloca en el tope y es la primera en ser retirada al deshacer.
2. **Las personas esperan su turno en una fila de atención y se atiende primero a quien llegó primero:**
    - **Estructura:** Cola (FIFO).
    - **Justificación:** Cumple la regla *First-In, First-Out* (primero en entrar, primero en salir), respetando el orden de llegada.
3. **Una aplicación recorre las canciones de una lista y, al terminar la última, vuelve automáticamente a la primera:**
    - **Estructura:** Lista circular simple.
    - **Justificación:** El último nodo apunta directamente al primer nodo en lugar de a nulo, permitiendo la repetición continua del ciclo.
4. **Un sistema necesita recorrer una colección de registros en un único sentido y agregar elementos al comienzo con frecuencia:**
    - **Estructura:** Lista simplemente enlazada (Lista simple).
    - **Justificación:** Utiliza un único enlace siguiente por nodo y permite realizar inserciones al comienzo de forma eficiente en tiempo constante O(1).
5. **Un reproductor permite pasar a la canción siguiente y a la anterior desde cualquier canción:**
    - **Estructura:** Lista doblemente enlazada (Lista doble).
    - **Justificación:** Posee referencias a siguiente y anterior en cada nodo, haciendo posible la navegación en ambas direcciones.

Parte 2 — Pilas (LIFO)1

2.1 Sigue las operaciones

Secuencia de ejecuciones sobre la pila inicialmente vacía1:

1. push(4) ⟹ Pila: (Tope: 4)
2. push(9) ⟹ Pila: (Tope: 9)
3. push(2) ⟹ Pila: 1 (Tope: 2)
4. pop() ⟹ Retira y devuelve **2**. Pila: (Tope: 9)
5. push(7) ⟹ Pila: (Tope: 7)
6. peek() ⟹ Consulta y devuelve **7**. Pila no cambia: ` `
7. pop() ⟹ Retira y devuelve **7**. Pila: (Tope: 9)
8. pop() ⟹ Retira y devuelve **9**. Pila: (Tope: 4)
9. **Resultado final:** En la pila queda únicamente el elemento **4**, que es también el **tope**.
10. **Representación del estado final:**

    [Tope] -> | 4 | <- [Base]

2.2 Casos límite

- **pop() en pila vacía:** Ocurre un desbordamiento inferior (underflow). Se maneja validando si la pila está vacía antes de retirar, retornando un valor especial (nulo) o lanzando una excepción informativa.
- **peek() en pila vacía:** Devuelve nulo o genera una excepción indicando la ausencia de elementos.
- **Pseudocódigo deshacer(pila):**

  Funcion deshacer(pila)

    Si pila.estaVacia() Entonces

      Escribir "No hay acciones para deshacer"

      Retornar nulo

    Sino

      Retornar pila.pop()

    FinSi

  FinFuncion

  2.3 Aplicación: deshacer acciones

  Secuencia realizada: escribir ⟹ borrar ⟹ pegar.

- **Antes del primer deshacer:** [escribir, borrar, pegar] (Tope: pegar).
- **Después del 1.er deshacer:** Se retira **pegar**. Pila queda: [escribir, borrar] (Tope: borrar).
- **Después del 2.º deshacer:** Se retira **borrar**. Pila queda: [escribir] (Tope: escribir).
- **Análisis:** Se deshace primero la acción **pegar**. Esto ocurre porque fue la última acción ingresada y, por la regla LIFO de la pila, se encuentra en el tope.

3.1 Simula la fila de atención

Secuencia sobre la cola inicialmente vacía1:

1. encolar("Ana") ⟹ Cola: [Ana] (Frente: Ana, Final: Ana)
2. encolar("Luis") ⟹ Cola: [Ana, Luis] (Frente: Ana, Final: Luis)
3. encolar("Sol") ⟹ Cola: [Ana, Luis, Sol] (Frente: Ana, Final: Sol)
4. desencolar() ⟹ Devuelve **"Ana"**. Cola: [Luis, Sol] (Frente: Luis, Final: Sol)
5. encolar("Mia") ⟹ Cola: [Luis, Sol, Mia] (Frente: Luis, Final: Mia)
6. desencolar() ⟹ Devuelve **"Luis"**. Cola: [Sol, Mia] (Frente: Sol, Final: Mia)
7. encolar("Teo") ⟹ Cola: [Sol, Mia, Teo] (Frente: Sol, Final: Teo)
8. **Resultado:** Quedan esperando **Sol, Mia y Teo**, en ese orden específico (Frente: Sol, Final: Teo)

3.2 Operaciones y casos límite

- **Pseudocódigo atender(cola):**

```
Funcion atender(cola)
  Si cola.estaVacia() Entonces
    Escribir "No hay personas en espera"
    Retornar nulo
  Sino
    Retornar cola.desencolar()
  FinSi
FinFuncion
```

- **Manejo de cola vacía al desencolar:** Controlar la operación mediante una verificación previa de estaVacia() para prevenir accesos a referencias nulas.
- **Agregar al frente y retirar del final:** Aunque lógicamente mantiene un orden de salida, alterar el punto de inserción y extracción sin invertir los punteros rompería la eficiencia O(1) de la cola enlazada si se carece de punteros dobles.

Parte 4 — Listas enlazadas simples

4.1 Construye y modifica una lista

Evolución de la lista paso a paso:

1. Insertar 18 al final: Cabeza -> -> nulo
2. Insertar 7 al comienzo: Cabeza -> -> -> nulo
3. Insertar 24 al final: Cabeza -> -> -> -> nulo
4. Insertar 13 después de 7: Cabeza -> -> -> -> -> nulo
5. Eliminar 18: Cabeza -> -> -> -> nulo

4.2 Diseña operaciones

```
Funcion recorrer(cabeza)
  actual = cabeza
  Mientras actual != nulo Hacer
    Imprimir actual.dato
    actual = actual.siguiente
  FinMientras
FinFuncion

Funcion buscar(cabeza, valor)
  actual = cabeza
  Mientras actual != nulo Hacer
    Si actual.dato == valor Entonces
      Retornar Verdadero
    FinSi
    actual = actual.siguiente
  FinMientras
  Retornar Falso
FinFuncion

Funcion insertarAlComienzo(cabeza, valor)
  nuevoNodo = Nuevo Nodo(valor)
  nuevoNodo.siguiente = cabeza
  cabeza = nuevoNodo
  Retornar cabeza
FinFuncion
```

4.3 Comprueba los bordes

- **Lista vacía vs. lista con nodos al insertar al comienzo:** Si está vacía (cabeza == nulo), nuevoNodo.siguiente recibe nulo y cabeza apunta al nuevo nodo. Si ya tiene nodos, nuevoNodo.siguiente apunta al nodo que era la cabeza previa. Ambas situaciones se ejecutan con la misma lógica sin requerir condicionales especiales.
- **Eliminar el primer nodo:** Se actualiza cabeza = cabeza.siguiente

Parte 5 — Listas circulares

5.1 Reproduce un recorrido circular

- Lista inicial: P1 -> P2 -> P3 -> P1 (enlace del último al primero).
- **Registro de turnos (P2 termina en su 2.º turno):**
    1. Turno 1: Atiende **P1** -> Siguiente: P2
    2. Turno 2: Atiende **P2** (1.er turno de P2) -> Siguiente: P3
    3. Turno 3: Atiende **P3** -> Siguiente: P1
    4. Turno 4: Atiende -> Siguiente: P2
    5. Turno 5: Atiende **P2** (2.º turno de P2, **termina y se retira**) -> Siguiente: P3
    6. Turno 6: Atiende **P3** -> Siguiente: P1
    7. Turno 7: Atiende **P1** -> Siguiente: P3
- **Lista resultante:** P1 -> P3 -> P1 (P3 apunta directamente a P1).
- **Condición de parada de recorrido:** Almacenar la referencia al nodo inicial en una variable inicio y detener la iteración cuando actual.siguiente == inicio

5.2 Casos especiales

- **Un solo nodo:** Al eliminarlo, la referencia cabeza pasa a ser nulo.
- **Eliminar el primer nodo:** La cabeza pasa al nodo siguiente, y se actualiza la referencia siguiente del último nodo para que apunte a la nueva cabeza.

Parte 6 — Listas doblemente enlazadas1

6.1 Recorrido en ambos sentidos

- **Estructura:** nulo <- [Lunes] ⇄ [Martes] ⇄ [Miércoles] ⇄ [Jueves] -> nulo.
- **Recorrido inicio** -> **final:** Lunes, Martes, Miércoles, Jueves.
- **Recorrido final** ->**inicio:** Jueves, Miércoles, Martes, Lunes.

6.2 Inserta y elimina

1. Insertar Domingo antes de Lunes: Domingo ⇄ Lunes ⇄ Martes ⇄ Miércoles ⇄ Jueves
2. Insertar Viernes después de Jueves: Domingo ⇄ Lunes ⇄ Martes ⇄ Miércoles ⇄ Jueves ⇄ Viernes
3. Eliminar Miércoles: Domingo ⇄ Lunes ⇄ Martes ⇄ Jueves ⇄ Viernes
    - Ajuste de enlaces: Martes.siguiente apunta a Jueves y Jueves.anterior apunta a Martes

6.3 Compara los enlaces

- **Ventaja:** Permite recorridos bidireccionales y la eliminación directa de un nodo conocido en **O(1)** sin tener que buscar el nodo previo.
- **Costo:** Ocupa más memoria al almacenar dos referencias (anterior y siguiente) por nodo y requiere mantener actualizados ambos punteros en cada operación

Parte 7 — Análisis y elección

Estructura y representación	Operación	Complejidad	Justificación breve
Pila enlazada con referencia al tope	Insertar en el tope	O(1)	Acceso e inserción directa en el tope1.
Pila enlazada con referencia al tope	Retirar del tope	$O(1)$	Se actualiza la referencia del tope en un paso1.
Cola enlazada con referencias al frente y al final	Encolar al final	$O(1)$	Se inserta directamente usando el puntero final1.
Cola enlazada con referencias al frente y al final	Desencolar del frente	$O(1)$	Se remueve directamente usando el puntero frente1.
Lista simple sin referencia al último nodo	Insertar al comienzo	$O(1)$	Se actualiza la cabeza en tiempo constante1.
Lista simple sin referencia al último nodo	Insertar al final	O(n)	Debe recorrer los n nodos hasta hallar el final1.
Lista doble con referencia a un nodo conocido	Desconectar ese nodo	O(1)	Se ajustan las referencias de los vecinos directamente

¿Por qué una cola sin referencia al final tarda más? Porque para agregar un elemento al final debe iterar a través de toda la lista desde la cabeza hasta encontrar el último nodo (O(n)).

Ventaja de la lista circular para reiniciar recorridos: El último nodo enlaza con el primero, permitiendo regresar al inicio de forma transparente con un simple avance siguiente

Parte 8 — Reflexión

La forma en que se conectan los nodos determina directamente el orden de inserción y extracción de datos y la eficiencia operacional de la estructura. Por ejemplo, en una **pila**, la restricción de acceder únicamente por un extremo genera una disciplina LIFO, ideal para deshacer operaciones. En contraste, en una **cola**, conectar la inserción al final y la extracción al frente establece un orden FIFO, esencial para gestionar turnos. Por su parte, la **lista circular** conecta el último nodo de regreso al primero para permitir ciclos continuos de ejecución. Así, la presencia de enlaces simples, dobles o circulares transforma la manera en que fluyen los datos en la memoria.